Modelli e algoritmi per il supporto delle decisioni

Dmitri Ollari Ischimji

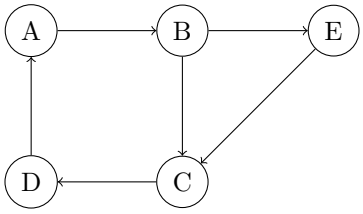
24 giugno 2023

Indice

Capitolo 1

Introduzione ai grafi

1.1 Grafo



Un grafo G è costruti da una coppia di insieme (V, A), dove V è l'insieme dei vertici e A è l'insieme degli archi. Esistono due tipi di grafi:

- **Grafo orientato**: gli archi sono orientati, ovvero $(u, v) \neq (v, u)$.
- **Grafo non orientato**: gli archi non sono orientati, ovvero $(u, v) = (v, u)$.

1.1.1 Liste di adiacenza

Per rappresentare un grafo si possono utilizzare le liste di adiacenza, un esempio è il seguente:

Nodo	Adiacenti
A	B, D
B	A, C
C	B, D
D	A, C

1.1.2 Matrice di adiacenza

Se il grafo è **orientato** la matrice di adiacenza è la seguente:

	A	B	C	D
A	0	1	0	1
B	0	0	1	0
C	0	0	0	1
D	1	0	0	0

Le righe rappresentano i nodi di partenza degli archi, mentre le colonne rappresentano i nodi di arrivo. Gli elementi della matrice indicano se c'è un arco che collega i nodi (1 se c'è un arco, 0 altrimenti).

Se il grafo è **non orientato** la matrice di adiacenza è la seguente:

	A	B	C	D
A	0	1	0	1
B	1	0	1	0
C	0	1	0	1
D	1	0	1	0

Le righe e le colonne rappresentano i nodi del grafo, mentre gli elementi della matrice indicano se due nodi sono adiacenti (1 se sono adiacenti, 0 altrimenti).

1.1.3 Cammino

Un cammino è una sequenza di vertici e di archi che collegano due vertici. Esistono diverse tipologie di cammini:

- **Cammino semplice**: un cammino in cui non ci sono archi percosi più di una volta.
- **Cammino elementare**: un cammino in cui non un nodo non è percorso più di una volta.

1.1.4 Ciclo

Un ciclo è un cammino in cui il nodo di partenza è uguale al nodo di arrivo.

1.1.5 Circuiti Hamiltoniani

Un circuito Hamiltoniano è un ciclo che passa per tutti i nodi del grafo una e una sola volta.

1.1.6 Componenti connesse

Dati due nodi u e v di un grafo G, si dice che u e v sono connessi se esiste un cammino che collega u e v.

1.1.7 Grafo completo

Un grafo è completo se ogni coppia di nodi è collegata da un arco.

1.1.8 Matching

Un matching è un sottoinsieme di archi di un grafo tale che ogni nodo è collegato ad al massimo un arco del matching.

1.1.9 Grafo bipartito

Un grafo è bipartito se i suoi nodi possono essere divisi in due insiemi V_1 e V_2 tali che ogni arco del grafo collega un nodo di V_1 con un nodo di V_2.

1.1.10 Albero

Un albero è un grafo connesso e aciclico.

Albero di supporto

Un albero di supporto di un grafo G è un sottografo di G che è un albero e che contiene tutti i nodi di G.

Capitolo 2

Minimum Spanning Tree

AKA Alberi di supporto a peso minimo.

2.1 Algoritmo greedy

L'algoritmo Greedy per il Minimum Spanning Tree (MST) è un algoritmo che trova il sottoinsieme di archi di peso minimo che connette tutti i nodi di un grafo non orientato. L'algoritmo si basa sull'idea di selezionare ripetutamente l'arco di peso minimo tra due nodi che non siano ancora collegati.

2.1.1 Passi dell'algoritmo

1. Inizializzare l'MST come un insieme vuoto di archi.
2. Ordinare gli archi del grafo in ordine crescente di peso.
3. Selezionare l'arco di peso minimo e controllare se collega due nodi diversi nell'MST corrente. Se sì, aggiungerlo all'MST.
4. Ripetere il passo precedente fino a quando tutti i nodi non sono connessi nell'MST.

2.1.2 Pseudocodice

Ecco il pseudocodice dell'algoritmo Greedy per MST:

```
GreedyMST(G):
  MST = {} // Inizializza l'MST come un insieme vuoto di archi
  Ordina gli archi di G in ordine crescente di peso
  Per ogni arco (u, v) in G:
    Se u e v non sono nello stesso componente connesso di MST:
      Aggiungi (u, v) all'MST
      Unisci i componenti connessi di u e v in MST
  Ritorna MST
```

2.1.3 Complessità

L'algoritmo Greedy per MST ha una complessità temporale di $O(E \log E)$, dove E è il numero di archi nel grafo.

2.1.4 Correttezza

L'algoritmo Greedy per MST è basato sull'idea di selezionare ripetutamente l'arco di peso minimo tra due nodi che non siano ancora collegati nell'MST corrente. L'obiettivo dell'algoritmo è quello di costruire un MST che connetta tutti i nodi del grafo, utilizzando gli archi di peso minimo possibili.

La correttezza dell'algoritmo può essere dimostrata utilizzando l'argomento del sottografo in espansione. Inizialmente, l'MST è un insieme vuoto di archi. Ad ogni passo, l'algoritmo seleziona l'arco di peso minimo che collega due nodi diversi nell'MST corrente. Questo arco viene aggiunto all'MST, espandendo così il sottografo. L'algoritmo continua a selezionare e aggiungere gli archi di peso minimo fino a quando tutti i nodi non sono connessi nell'MST.

La dimostrazione della correttezza si basa su due proprietà fondamentali:

1. L'arco di peso minimo tra due nodi che non sono ancora collegati nell'MST corrente è sicuramente parte del MST finale. Questo perché, se ci fosse un MST diverso che non contiene quell'arco, potremmo sostituire un arco nell'MST con l'arco di peso minimo per ottenere un MST con un peso minore, il che contraddice l'ipotesi che l'MST sia il più leggero possibile.

2. L'aggiunta di un arco all'MST non crea cicli. Questo perché l'algoritmo controlla se i due nodi dell'arco selezionato sono già connessi nello stesso componente connesso dell'MST. Se fossero già connessi, l'aggiunta di quell'arco creerebbe un ciclo nel grafo, quindi l'arco non viene aggiunto.

Basandosi su queste proprietà, l'algoritmo Greedy per MST seleziona e aggiunge gli archi di peso minimo in modo che l'MST risultante sia un albero senza cicli che connette tutti i nodi del grafo originale. Pertanto, l'algoritmo produce un MST corretto per il grafo di input.

Capitolo 3

Matching

3.1 Definizioni

Il matching, nell'ambito della teoria dei grafi, è un concetto che si applica ai grafi bipartiti. Un matching in un grafo bipartito è un sottoinsieme degli archi del grafo tale che nessun paio di archi nel matching condivida lo stesso nodo di partenza o di arrivo. In altre parole, un matching è un insieme di archi che non si sovrappongono tra loro.

Formalmente, dato un grafo bipartito $G = (V, E)$ con partizione dei nodi in due insiemi disgiunti V_1 e V_2, un matching M in G è un sottoinsieme di E tale che per ogni coppia di archi $e_1, e_2 \in M$, i nodi di partenza o di arrivo di e_1 e e_2 non coincidano.

3.2 Matching di peso massimo

Il "matching di peso massimo" è un tipo particolare di matching in cui ogni arco ha un peso associato e l'obiettivo è trovare un matching che massimizzi la somma dei pesi degli archi selezionati.

Formalmente, dato un grafo bipartito ponderato $G = (V, E, w)$ con partizione dei nodi in due insiemi disgiunti V_1 e V_2, dove $w : E \to \mathbb{R}$ è una funzione che assegna un peso reale a ogni arco, il matching di peso massimo è un sottoinsieme $M \subseteq E$ tale che la somma dei pesi degli archi in M sia massima rispetto a tutti gli altri matching possibili in G.

3.3 Metching di cardinalità massima

Questo è un tipo particolare di matching in cui ogni arco ha un peso **unitario** associato e l'obiettivo è trovare un matching che massimizzi il numero di archi selezionati.

3.4 Matching initale

Il "matching iniziale" è un concetto che si applica alla ricerca di un matching in un grafo bipartito. Indica un sottoinsieme di archi che viene selezionato come punto di partenza per l'algoritmo di ricerca del matching ottimale.

Si ponga $M = \emptyset$.

1. Se esiste un arco $e \in A \setminus M$ che non sia adiacente ad alcun arco in M, allora porre $M = M \cup \{e\}$ e ripetere il passo 1.
2. Altrimenti: STOP.

3.5 Algoritmo con nome non specificato dal prof :(

Sia M un matching di partenza (eventualmente individuato con la procedura vista).

Passo 0: Tutti i vertici del grafo sono non etichettati.

Passo 1: Se non c'è alcun vertice del grafo in V_1 che soddisfa le seguenti proprietà:

- È non etichettato.
- Su di esso non incide alcun arco in M.

Allora: STOP. Altrimenti, si seleziona un vertice del grafo in V_1 con tali proprietà e gli si assegna l'etichetta $(E, -)$. Si inizializza l'insieme R dei vertici analizzati con $R = \emptyset$.

Passo 2: Se tutti i vertici etichettati sono stati analizzati, si ritorna al Passo 1. Altrimenti, si seleziona un vertice k etichettato ma non analizzato ed lo si analizza, cioè si pone $R = R \cup \{k\}$. Analizzare un vertice significa compiere le seguenti operazioni:

a) Se la prima componente dell'etichetta di k è una E, allora si assegna un'etichetta (O, k) a tutti i vertici del grafo adiacenti a k e non ancora etichettati. Quindi si ripete il Passo 2.

b) Se la prima componente dell'etichetta di k è una O, allora sono possibili due casi:

- **Caso 1:** C'è un arco marcato incidente su k. In tal caso, si assegna l'etichetta (E, k) al vertice unito a k dall'arco in M e si ripete il Passo 2.
- **Caso 2:** Non ci sono archi in M incidenti su k. In tal caso, si passa al Passo 3.

Passo 3: Utilizzando la seconda componente delle etichette, si risale dal vertice k in cui ci si è bloccati al Passo 2 fino al vertice s con seconda componente pari a $-$. In questo modo si individua un cammino da s a k che inizia con un arco non appartenente a M, prosegue alternando archi in M e archi non appartenenti a M, e termina in k con un arco non appartenente a M. A questo punto, si aggiorna M invertendo l'appartenenza e non appartenenza a M per i soli archi lungo tale cammino. Quindi, tutti gli archi non appartenenti a M lungo il cammino vengono inseriti in M e viceversa. Infine, si cancellano tutte le etichette ripartendo dal Passo 1.

3.5.1 Complessità

$O(|V|^{\frac{1}{2}}|A|)$

Capitolo 4

Flusso

4.1 Problemi di flusso a costo minimo

4.1.1 Ammissibilità e degenerazione

Se i flussi associati agli archi in base hanno valore **non negativo**, si parla di **soluzione ammissibile**.

Se uno o più flussi associati agli archi in base hanno valore **nullo**, si parla di **soluzione degenere**.

4.1.2 Coefficienti di costo ridotto

Ad ogni arco fuori base è assocaito un **coefficiente di costo ridotto** che misura la variazione del valore dell'obiettivo al crescere di una unità del flusso associato all'arco fuori base.

4.1.3 Condizione di ottimalità

Se i coefficienti di costo ridotto di tutti gli archi fuori base sono **non negativi**, in tal caso la soluzione di base attuale è **ottima**.

Calcolo del costo ridotto

1. Prendere un arco fuori base.
2. Aggiungere l'arco all'albero di supporto corrispondente alla base attuale.
3. Considerare l'unico ciclo che si forma con tale aggiunta.
4. Fissare come verso del ciclo quello dell'arco fuori base.
5. Calcolare il coefficiente di costo ridotto sommando tra loro tutti i costi relativi agli archi attraversati dal ciclo nel loro stesso verso e sottraendo al risultato i costi degli archi attraversati dal ciclo in senso opposto.

4.1.4 Condizione di illimitatezza

Se l'aggiunta all'albero di supporto/base attuale di un arco fuori base con coefficiente di costo ridotto negativo crea un ciclo orientato, allora il problema di flusso a costo minimo ha obiettivo illimitato.

4.1.5 Cambio di base

Se le condizioni di ottimalità e illimitatezza non sono soddisfatte si procede con il cambio di base scegliendo un'arco da inserire e un'arco da rimuovere dalla base.

Per gli archi entranti in base bisogna scegliere quelli con coefficiente di costo ridotto negativo.

4.1.6 Algoritmo del simplesso su rete

Una volta completate le operazioni relative a una base e definita una nuova base, l'algoritmo del simplesso su rete ripete le operazioni appena viste sulla nuova base, cioè:

1. Verifica di ottimalità.
2. Verifica di illimitatezza.
3. Cambio di base.

E itera questo procedimento fino a quando arriva a una base che soddisfi la condizione di ottimalità o quella di illimitatezza.

4.2 Problemi di flusso massimo

Vogliamo determinare la quantità massima di flusso che partendo dal nodo sorgente si può far giungere fino al nodo destinazione, tenuto conto dei vincoli di capacità sugli archi e di quelli di equilibrio nei nodi intermedi.

4.2.1 Tagli nella rete

Considero $U \subset V$ dove V rappresenta l'insieme dei vertici e aggiungo due proprietà:

- $S \in U$, S è il nodo sorgente
- $D \notin U$, D è il nodo destinazione

Posso separare il grafo di partenza nei due insieme U e $V \setminus U$.
Il taglio è l'insieme degli archi che collegano i due insiemi.

4.3 Problema di taglio minimo

4.3.1 Archi saturi

Supponiamo di avere un flusso ammissibile $X = (x_{ij})$ per cui sono soddisfatti i vincoli di equilibrio e quelli di capacità degli archi. Se $x_{ij} = c_{ij}$, diremo che l'arco (i, j) è saturo.

Dove con c_{ij} si intende la capacità dell'arco (i, j).

4.3.2 Cammini privi di archi saturi

Se un cammino dal nodo S al nodo D non presenta nessun arco saturo, il flusso X non è ottimo.

Possiamo aumentare il flusso lungo ciascun arco del cammino di una quantità Δ definita nel seguente modo:

$$\Delta = \min_{i=0,\dots,r} \left[c_{q_i q_{i+1}} - x_{q_i q_{i+1}}\right] > 0,$$

senza violare i vincoli di capacità degli archi.

Si noti anche che i vincoli di equilibrio continuano ad essere rispettati, poiché in ogni nodo intermedio del grafo facente parte del cammino, il flusso in entrata aumenta di Δ, ma contemporaneamente aumenta anche il flusso in uscita di Δ.

4.3.3 Grafo associato ad un flusso ammissibile

Definisco un nuovo grafo orientato $G(X) = (V, A(X))$.

Il nuovo grafo ha gli stessi nodi della rete originaria e ha il seguente insieme di archi:

$$A(X) = \{(i, j) : (i, j) \in A, x_{ij} < c_{ij}\} \cup \{(i, j) : (j, i) \in A, x_{ji} > 0\}$$

L'insieme $A(X)$ contiene tutti gli archi di A che non sono saturi (l'insieme $A_f(X)$, detto insieme degli archi forward) e tutti gli archi di A lungo cui è stata inviata una quantità positiva di flusso, ma invertiti di verso (l'insieme $A_b(X)$, detto insieme degli archi backward).

- Per gli archi **forward**, il valore α_{ij} rappresenta quanto flusso posso ancora inviare lungo l'arco $(i, j) \in A$ della rete originaria.
- Per gli archi **backward**, il valore α_{ij} rappresenta quanto flusso posso rispedire indietro lungo l'arco $(j, i) \in A$ della rete originaria.

4.3.4 Aggiornamento del flusso

Per prima cosa devo trovare il Δ di un cammino, si deve prendere il valore minore di un cammino.

Dopo di che posso aggionrare il flusso con la seguente formula:

$$x_{ij} = \begin{cases} x_{ij} + \Delta & \text{se } (i, j) \in A_f(X) \cap P \\ x_{ij} - \Delta & \text{se } (j, i) \in A_b(X) \cap P \\ x_{ij} & \text{altrimenti} \end{cases} \tag{4.1}$$

L'aggiornamento del flusso garantisce che questo sia ancora ammissibile, cioè soddisfa i vincoli di non negatività delle variabili, i vincoli di capacità sugli archi e i vincoli di equilibrio nei nodi intermedi.

Nella rete originaria, incrementiamo di Δ il flusso lungo gli archi corrispondenti ad archi forward del grafo associato al flusso attuale, appartenenti al cammino P. Rispediamo indietro Δ unità di flusso lungo gli archi che, cambiati di

verso, corrispondono ad archi backward del grafo associato, appartenenti al cammino P, e quindi decrementiamo il flusso della stessa quantità lungo tali archi. Infine, lungo gli archi che non fanno parte del cammino P, il flusso rimane invariato.

Il nuovo flusso è ammissibile e la quantità di flusso uscente da S è anch'essa aumentata proprio della quantità Δ, il che dimostra che il flusso precedente non era ottimo.

4.4 Algoritmo di Ford-Fulkerson per il flusso massimo

Passo 0 - Inizializzazione

È dato un flusso ammissibile iniziale X (si noti che si può sempre partire con il flusso nullo $X = 0$).

Passo 1

Si costruisca il grafo $G(X)$ associato al flusso attuale X.

Passo 2

Se non esiste alcun cammino orientato da S a D in $G(X)$, allora STOP: il flusso attuale X è ottimo. Altrimenti, si individui un tale cammino P e si aggiorni il flusso X come visto in precedenza e si ritorni al Passo 1.

4.5 Procedura di etichettatura

Passo 0

Si parta con un flusso ammissibile X. Si noti che è sempre possibile partire con il flusso nullo $X = 0$.

Passo 1

Si associ alla sorgente S l'etichetta (S, ∞). L'insieme R dei nodi analizzati è vuoto, ovvero $R = \emptyset$, mentre l'insieme E dei nodi etichettati contiene il solo nodo S, ovvero $E = \{S\}$.

Passo 2

Se $E \setminus R = \emptyset$, il flusso attuale X è ottimo. Altrimenti, si selezioni un nodo $i \in E \setminus R$, cioè etichettato, con etichetta (k, Δ), ma non ancora analizzato, e lo si analizzi. Per ogni nodo $j \notin E$, cioè non ancora etichettato, e tale che $(i, j) \in A(X)$, si etichetti j con la seguente etichetta:

$$(i, \min(\Delta, c_{ij} - x_{ij})) \text{ se } (i, j) \in A_f(X) \tag{4.2}$$

$$(i, \min(\Delta, x_{ji})) \text{ se } (i, j) \in A_b(X) \tag{4.3}$$

Quindi si ponga:

$$E = E \cup \{j \notin E : (i, j) \in A(X)\} \tag{4.4}$$

$$R = R \cup \{i\} \tag{4.5}$$

Se $D \in E$, cioè se la destinazione è stata etichettata, si vada al Passo 3, altrimenti si ripeta il Passo 2.

Passo 3

Si ricostruisca un cammino orientato da S a D in $G(X)$ procedendo a ritroso da D verso S ed usando le prime componenti delle etichette. A questo punto il cammino trovato

$$S \to q_1 \to \ldots \to q_{r-1} \to q_r \to D \tag{4.6}$$

con insieme di archi

$$P = \{(S, q_1), \ldots, (q_{r-1}, q_r), (q_r, D)\} \tag{4.7}$$

è un cammino orientato da S a D in $G(X)$. Ora possiamo aggiornare X come visto in precedenza, dove il valore Δ è dato dalla seconda componente dell'etichetta del nodo D, e ritornare al Passo 1.

Capitolo 5

Assegnamento

Sia dato un grafo bipartito completo $G = (V_1 \cup V_2, E)$, nel problema di assegnamento si cerca di effettare un matching fra i due sotto insieme di vertici di modo da ottenere il costo minore possbili per ogni accoppiamento.

	Job 1	Job 2	Job 3	Job 4
Worker 1	10	8	9	7
Worker 2	6	9	7	5
Worker 3	8	7	6	9
Worker 4	5	6	7	8

5.1 Algoritmo ungherese

5.1.1 Riduzione

Il primo passo dell'algoritmo ungherese è quello di ridurre al minimo la matrice dei costi e si porsegue nella seguente maniera:

1. Per ogni riga, sottrarre il minimo valore della riga a tutti i valori della riga.
2. Per ogni colonna, sottrarre il minimo valore della colonna a tutti i valori della colonna.

Applicando la riduzione alla matrice dei costi si ottiene:
Sottrai il valore minimo di ogni riga dalla riga corrispondente:

1. Sottrai 7 da ogni valore nella riga del Worker 1
2. Sottrai 5 da ogni valore nella riga del Worker 2
3. Sottrai 6 da ogni valore nella riga del Worker 3
4. Sottrai 5 da ogni valore nella riga del Worker 4

	Job 1	Job 2	Job 3	Job 4
Worker 1	3	1	2	0
Worker 2	1	4	2	0
Worker 3	2	1	0	3
Worker 4	0	1	2	3

1. Sottrai 0 da ogni valore nella colonna del Job 1
2. Sottrai 1 da ogni valore nella colonna del Job 2
3. Sottrai 0 da ogni valore nella colonna del Job 3
4. Sottrai 0 da ogni valore nella colonna del Job 4

	Job 1	Job 2	Job 3	Job 4
Worker 1	3	0	2	0
Worker 2	1	3	2	0
Worker 3	2	0	0	3
Worker 4	0	0	2	3

5.1.2 Fase di assegnazione

Dopo aver ridotto la matrice, cerco di coprire con varie righe verticali ed orizzontali la matrice, le righe vanno messe dove sono gli zeri all'interno della matrice ridotta.

Se riesco a coprire tutti gli zeri con n linee, dove n rappresenta il numero di righe e colonne della matrice ridotta, possono selezionare i vari elementi dove sono preseti gli zeri e questi rappresentano l'assegnamento ottimo. Al che seleziono i valori della matrice originale in corrispondenza degli zeri selezionati nella matrice ridotta.

Se non dovessi avere abbastanza zeri da poter coprire tutta la matrice, allora devo procedere con la fase di correzione.

Si seleziona l'elemento più piccolo diverso da zero della riga o colonna incriminata e si sottrae a tutti gli elementi della riga o colonna, mentre si aggiunge il valore selezionato nella cella dove è presente un incrocio di riga e colonna.

Capitolo 6

Branch and Bound

L'algoritmo di Branch and Bound è un metodo di risoluzione esatto utilizzato per problemi di ottimizzazione, in cui si cerca di trovare la soluzione migliore tra un insieme di possibili soluzioni. L'algoritmo si basa sulla suddivisione del problema in sotto-problemi più piccoli, chiamati "branch", e sulla valutazione di limiti superiori e inferiori per determinare quali sotto-problemi esplorare ulteriormente.

L'algoritmo di Branch and Bound può essere descritto in modo generico come segue:

1. Inizializzazione: Si parte da una soluzione iniziale e si calcola un lower bound e un upper bound per il problema.
2. Esplorazione dei sotto-problemi: Si seleziona un sotto-problema da esplorare. Questo può essere fatto in base a una strategia specifica, come ad esempio la selezione del sotto-problema con l'upper bound più basso.
3. Generazione dei sotto-problemi: Si generano i sotto-problemi a partire dal sotto-problema selezionato, suddividendo il problema originale in problemi più piccoli. Questo può essere fatto introducendo vincoli aggiuntivi o modificando i parametri del problema.
4. Calcolo dei limiti superiori e inferiori: Per ogni sotto-problema generato, si calcolano i limiti superiori e inferiori. L'upper bound rappresenta un limite superiore per il valore ottimo del sotto-problema, mentre il lower bound rappresenta un limite inferiore.
5. Pruning: Si eliminano i sotto-problemi che possono essere esclusi in base ai limiti superiori e inferiori calcolati. Se l'upper bound di un sotto-problema è inferiore al lower bound di un altro sotto-problema già esplorato, il primo sotto-problema può essere eliminato, poiché non può contenere una soluzione ottima.
6. Aggiornamento della soluzione ottima: Durante l'esplorazione dei sotto-problemi, si tiene traccia della soluzione ottima finora trovata.
7. Ripetizione: Si ripetono i passaggi 2-6 fino a quando non si esauriscono i sotto-pro

6.1 Upper bound

L'upper bound (limite superiore) è un valore che rappresenta un limite superiore per il valore ottimo del problema. In altre parole, è un valore che sicuramente non sarà inferiore alla soluzione ottima. L'upper bound viene utilizzato nell'algoritmo di Branch and Bound per determinare se un sotto-problema può essere eliminato o meno.

L'upper bound è vincolato da due caratteristiche:

- Tempi di calcolo molto inferiori rispetto ai tempi per la risoluzione del problema
- Valore molto vicino al valore ottimo

6.1.1 Rilassamento

Il rilassamento è una tecnica utilizzata per semplificare un problema originale, allentando alcune restrizioni o parametri. Nel contesto dell'algoritmo di Branch and Bound, il rilassamento viene utilizzato per determinare un upper bound per il valore ottimo del problema, calcolando la soluzione di un problema rilassato.

6.2 Lower bound

Il lower bound (limite inferiore) è un valore che rappresenta un limite inferiore per il valore ottimo del problema. In altre parole, è un valore che sicuramente non sarà superiore alla soluzione ottima. Il lower bound viene utilizzato nell'algoritmo di Branch and Bound per determinare se un sotto-problema può essere eliminato o meno.

6.3 Branching

Il branching è un'operazione utilizzata nell'algoritmo di Branch and Bound per generare nuovi sotto-problemi da esplorare. L'operazione di branching consiste nel dividere un sotto-problema T in due o più sotto-problemi più piccoli, chiamati nodi figli o successori di T. Il branching viene utilizzato per esplorare in modo più efficiente lo spazio delle soluzioni del problema, generando sotto-problemi più piccoli e quindi più facili da risolvere. Inoltre, il branching viene utilizzato per eliminare i sotto-problemi che non possono contenere la soluzione ottima del problema.

6.3.1 Cancellazione di sotto-problemi

Se l'Upper bound del sottoproblema è inferiore o uguale al Lower bound attuale, viene scartato il sotto problema.

Capitolo 7

Problema dello zaino

1. **Inizializzazione:** Si crea un nodo radice che rappresenta lo stato iniziale del problema, in cui nessun oggetto è ancora stato selezionato.
2. **Calcolo dell'upper bound:** Si calcola l'upper bound per il nodo corrente utilizzando una procedura specifica. Questo valore rappresenta il massimo valore che può essere ottenuto selezionando gli oggetti rimanenti nello zaino.
3. **Calcolo del lower bound:** Si calcola il lower bound per il nodo corrente utilizzando una procedura specifica. Questo valore rappresenta il minimo valore che può essere ottenuto selezionando gli oggetti rimanenti nello zaino.
4. **Pruning:** Se l'upper bound del nodo corrente è inferiore al lower bound di un nodo già visitato, allora si può eliminare il nodo corrente e tutti i suoi sottoalberi, in quanto non porteranno a una soluzione ottima.
5. **Espansione dei nodi:** Se l'upper bound del nodo corrente è maggiore o uguale al lower bound di un nodo già visitato, allora si espandono i nodi successivi generando tutti i possibili sottoinsiemi di oggetti che possono essere aggiunti allo zaino.
6. **Aggiornamento dell'upper bound:** Durante l'espansione dei nodi, se si trova un sottoinsieme di oggetti che ha un valore superiore all'upper bound corrente, si aggiorna l'upper bound con il nuovo valore.
7. **Selezione del prossimo nodo:** Si seleziona il prossimo nodo da esplorare in base a una strategia di selezione, come ad esempio il nodo con l'upper bound più alto.
8. **Ripetizione dei passi 2-7:** Si ripetono i passi 2-7 fino a quando non si esplorano tutti i nodi o si trova una soluzione ottima.

Capitolo 8

Programmazione dinamica - rifare

Capitolo 9

Complessità

9.1 P, NP e NP-completo

I problemi P, NP e NP-completi sono concetti fondamentali nella teoria della complessità computazionale.

Il problema P rappresenta la classe dei problemi che possono essere risolti in tempo polinomiale da un algoritmo deterministico. In altre parole, per un problema appartenente a P, esiste un algoritmo che può risolverlo in un tempo che cresce al più in modo polinomiale rispetto alla dimensione dell'input.

Il problema NP rappresenta la classe dei problemi per i quali una soluzione proposta può essere verificata in tempo polinomiale da un algoritmo deterministico. In altre parole, se si ha una soluzione proposta per un problema NP, è possibile verificarne la correttezza in un tempo che cresce al più in modo polinomiale rispetto alla dimensione dell'input.

I problemi NP-completi sono una sottoclasse dei problemi NP che sono considerati tra i più difficili da risolvere. Un problema NP-completo è un problema per il quale ogni altro problema in NP può essere ridotto in modo polinomiale ad esso. In altre parole, se si trovasse un algoritmo efficiente per risolvere un problema NP-completo, si potrebbe risolvere in modo efficiente tutti i problemi in NP.

Tuttavia, non si sa ancora se P è uguale a NP o se P è diverso da NP. Questa è una delle domande aperte più importanti nella teoria della complessità computazionale. La maggior parte degli esperti ritiene che P sia diverso da NP, ma finora non è stata trovata una prova definitiva.